

PROFESSIONAL TECHNICAL GUIDE

Networking for Cloud & DevOps Engineers

A step-by-step guide: from IP addressing and DNS to cloud VPC design, Kubernetes networking, troubleshooting and interview preparation.

CREATED BY

Mohit Gadilohar

LinkedIn: linkedin.com/in/mohit-gadilohar-devops

For Cloud Engineers, DevOps Engineers, SREs and Platform Engineers

Edition 1.0 | October 2026

Contents

1. Why Networking Matters . . . 5

Who this guide is for . . . 5

The learning path . . . 5

2. Network Models and Encapsulation . . . 6

Step 1: The OSI model . . . 6

Step 2: The TCP/IP model . . . 6

Step 3: Encapsulation, how data travels . . . 6

3. IP Addressing and Subnetting . . . 7

Step 1: IPv4 addresses . . . 7

Step 2: CIDR notation . . . 7

Step 3: Private address ranges (RFC 1918) . . . 7

Step 4: Worked subnetting example . . . 8

Step 5: Plan CIDR for the whole organisation . . . 8

Step 6: IPv6 in one paragraph . . . 8

4. Switching, ARP, VLANs and Routing . . . 9

Step 1: How a host decides where to send a packet . . . 9

Step 2: ARP, finding the MAC address . . . 9

Step 3: VLANs, logical segmentation . . . 9

Step 4: Routing tables . . . 9

Step 5: Static and dynamic routing . . . 10

5. TCP, UDP and Ports . . . 11

Step 1: Ports and sockets . . . 11

Step 2: TCP vs UDP . . . 11

Step 3: The TCP three-way handshake . . . 11

Step 4: Connection teardown and states . . . 12

6. DNS, the Name System . . . 13

Step 1: How a name is resolved . . . 13

Step 2: Record types . . . 13

Step 3: Cloud DNS services . . . 13

Step 4: Debugging DNS . . . 14

7. HTTP, TLS and Load Balancing . . . 15

Step 1: HTTP essentials . . . 15
Step 2: TLS, how encryption is established . . . 15
Step 3: Layer 4 vs Layer 7 load balancers . . . 16
Step 4: Health checks and algorithms . . . 16
8. NAT, Firewalls and Access Control . . . 17
Step 1: Network Address Translation . . . 17
Step 2: Stateful vs stateless filtering . . . 17
Step 3: Writing least-privilege rules . . . 17
9. Cloud Networking . . . 18
Step 1: Building blocks and vocabulary . . . 18
Step 2: Reference architecture, a multi-AZ VPC . . . 18
Step 3: Build it, in order . . . 19
Step 4: Connecting networks . . . 20
Step 5: A minimal Terraform VPC . . . 21
10. Kubernetes Networking . . . 22
Step 1: The Kubernetes network model . . . 22
Step 2: CNI plugins . . . 22
Step 3: Services, stable access to Pods . . . 22
Step 4: Following a request end to end . . . 23
Step 5: Ingress and Gateway API . . . 23
Step 6: NetworkPolicy, default deny . . . 23
11. Network Security and Zero Trust . . . 25
Step 1: Defence in depth . . . 25
Step 2: Zero trust principles . . . 25
Step 3: Secure remote access . . . 25
Step 4: Service mesh and mTLS . . . 25
Step 5: Visibility . . . 25
12. Troubleshooting Method and Toolkit . . . 26
Step 1: A repeatable method . . . 26
Step 2: The command toolkit . . . 26
Step 3: Reading the symptom . . . 27
Step 4: Walkthrough, "the app cannot reach the database" . . . 27
Step 5: Walkthrough, "the Kubernetes Service has no response" . . . 28
Step 6: Metrics worth alerting on . . . 28

13. Practice Labs and Review Checklists 29

Step 1: Hands-on labs 29

Step 2: Network design review checklist 29

Step 3: Common mistakes to avoid 29

Step 4: Where to go next 30

14. Glossary 31

15. Interview Questions and Answers 32

Networking fundamentals 32

Cloud networking 33

Kubernetes networking 33

Troubleshooting scenarios 34

CHAPTER 1

Why Networking Matters

Every cloud outage, slow deployment and failed health check eventually traces back to a network path. This guide gives you a working mental model, in the order you will need it.

Who this guide is for

This guide is written for cloud engineers, DevOps engineers, SREs and platform engineers who build, deploy and operate systems on AWS, Azure, Google Cloud or Kubernetes. You do not need a networking degree. You need to know what happens to a packet from the moment a user clicks a link to the moment your container answers.

The learning path

Each chapter builds on the previous one. Follow the order the first time, then use the guide as a reference.

Step	Chapter	You will be able to
1	Network models	Place any problem on the right layer
2	IP addressing & subnetting	Design CIDR ranges that never collide
3	Switching & routing	Read a route table and predict packet paths
4	TCP, UDP & ports	Explain timeouts, resets and connection limits
5	DNS	Debug name resolution inside and outside the cloud
6	HTTP, TLS & load balancing	Choose and configure L4 / L7 load balancers
7	NAT & firewalls	Write least-privilege security rules
8	Cloud networking	Build a multi-AZ VPC / VNet with private connectivity
9	Kubernetes networking	Trace traffic through Ingress, Service and Pod
10	Security & zero trust	Segment and encrypt traffic by default
11	Troubleshooting	Diagnose issues with a repeatable method
12	Practice & checklists	Apply Infrastructure as Code and review designs

> **How to use this guide**
>
> Run the commands in the examples on a Linux machine or cloud shell as you read. Networking is learned by observing real packets, not by memorising definitions.

CHAPTER 2

Network Models and Encapsulation

Layered models let you reason about one problem at a time. Learn the layers once and every tool makes sense.

Step 1: The OSI model

The OSI model splits communication into seven layers. Real systems use the simpler TCP/IP model, but OSI numbering is the shared vocabulary of the industry: when someone says "a layer 7 load balancer" they mean OSI layer 7.

Layer	Name	Job	Examples	Cloud example
7	Application	Protocols apps speak	HTTP, DNS, gRPC, SSH	Application Load Balancer, WAF
6	Presentation	Encoding, encryption	TLS, JSON, gzip	TLS termination
5	Session	Session state	RPC, sockets	Sticky sessions
4	Transport	End-to-end delivery	TCP, UDP, QUIC	Network Load Balancer, security group ports
3	Network	Addressing and routing	IP, ICMP, BGP	VPC route tables, gateways
2	Data link	Local delivery	Ethernet, ARP, VLAN	Virtual NIC, subnet
1	Physical	Bits on a medium	Fibre, copper, Wi-Fi	Provider data centre

Step 2: The TCP/IP model

The Internet runs on four layers. Map them to OSI like this: **Application** (OSI 5-7), **Transport** (OSI 4), **Internet** (OSI 3) and **Link** (OSI 1-2). Most engineers use both vocabularies interchangeably.

Step 3: Encapsulation, how data travels

When an application sends data, each layer wraps it with its own header. The receiver unwraps in reverse. This is why a firewall can filter on IP and port without ever reading your HTTP payload.

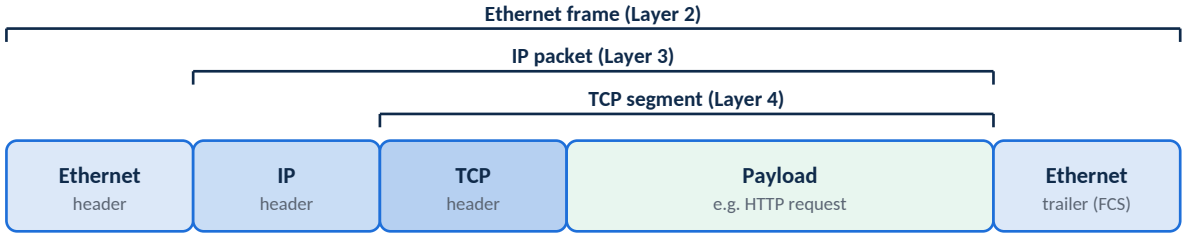


Figure 2.1: Encapsulation, each layer adds a header around the data from the layer above.

Why this matters in practice

Maximum Transmission Unit (MTU) limits the size of the Ethernet payload, usually 1500 bytes. Tunnels such as VPN, VXLAN and overlay networks add headers, shrinking usable space. Wrong MTU settings cause the classic symptom "small requests work, large uploads hang".

CHAPTER 3

IP Addressing and Subnetting

Addressing is the foundation of cloud design. A bad CIDR choice made on day one is painful to fix on day 500.

Step 1: IPv4 addresses

An IPv4 address is 32 bits, written as four decimal octets, for example `10.0.1.25`. Part of the address identifies the **network** and the rest identifies the **host**. The split is defined by the subnet mask or, in modern notation, the **CIDR prefix length**.

Step 2: CIDR notation

`10.0.0.0/16` means the first 16 bits are the network and the remaining 16 bits are for hosts. Larger prefix number means smaller network. The number of addresses is $2^{(32 - \text{prefix})}$.

CIDR	Subnet mask	Total addresses	Typical use
/16	255.255.0.0	65,536	Whole VPC / VNet
/20	255.255.240.0	4,096	Large subnet, Kubernetes nodes + pods
/24	255.255.255.0	256	Standard subnet
/26	255.255.255.192	64	Small tier subnet
/28	255.255.255.240	16	Smallest AWS subnet, endpoints
/32	255.255.255.255	1	A single host, firewall rule for one IP

> **Cloud providers reserve addresses**
>
> Traditional networks lose 2 addresses per subnet (network and broadcast). AWS and Azure reserve 5 per subnet and Google Cloud reserves 4. A /28 in AWS therefore gives 11 usable addresses, not 14.

Step 3: Private address ranges (RFC 1918)

These ranges are not routable on the public Internet. You use them inside VPCs, VNets and on-premises networks.

Range	CIDR	Addresses	Notes
10.0.0.0 - 10.255.255.255	10.0.0.0/8	16.7 million	Preferred for cloud, most room to plan
172.16.0.0 - 172.31.255.255	172.16.0.0/12	1 million	Docker default bridge uses 172.17.0.0/16
192.168.0.0 - 192.168.255.255	192.168.0.0/16	65,536	Common in home and office routers
100.64.0.0 - 100.127.255.255	100.64.0.0/10	4 million	Carrier-grade NAT, also used for extra pod ranges
169.254.0.0 - 169.254.255.255	169.254.0.0/16	65,536	Link-local, cloud metadata at 169.254.169.254

Step 4: Worked subnetting example

Question: you are given `192.168.10.0/26`. What are the network address, broadcast address and usable range?

1. Prefix is /26, so the host part is 32 - 26 = 6 bits. Total addresses = 2^6 = 64.
2. Mask: 26 network bits = 255.255.255.192. The block size in the last octet is 256 - 192 = 64.
3. Network address is the start of the block: **192.168.10.0**.
4. Broadcast address is the last address of the block: **192.168.10.63**.
5. Usable hosts are **192.168.10.1 - 192.168.10.62** (62 hosts in a classic network).
6. The next subnet of the same size starts at 192.168.10.64.

Step 5: Plan CIDR for the whole organisation

Plan address space once, centrally, before any team creates a VPC. Overlapping ranges make peering, VPN and Transit Gateway connections impossible without NAT workarounds.

Environment	Region	Planned VPC CIDR
Production	ap-south-1	10.10.0.0/16
Production	eu-west-1	10.11.0.0/16
Staging	ap-south-1	10.20.0.0/16
Development	ap-south-1	10.30.0.0/16
Shared services	ap-south-1	10.40.0.0/16
On-premises data centre	Mumbai	10.100.0.0/16
Reserved for growth	-	10.50.0.0 - 10.99.255.255

Step 6: IPv6 in one paragraph

IPv6 uses 128-bit addresses such as `2001:db8::1`. Subnets are normally /64. There is no NAT in the classic sense, so security is enforced by firewalls, not by address hiding. Clouds support dual-stack networks. Plan for it, but IPv4 remains the daily reality for most internal systems.

CHAPTER 4

Switching, ARP, VLANs and Routing

Layer 2 moves frames inside a subnet. Layer 3 moves packets between subnets. Knowing which one is failing saves hours.

Step 1: How a host decides where to send a packet

1. The host compares the destination IP with its own subnet using its mask.
2. If the destination is **in the same subnet**, the host sends the frame directly to the destination MAC address.
3. If it is **outside the subnet**, the host sends the frame to its **default gateway** (the router).
4. The router looks up the destination in its routing table and forwards the packet to the next hop.

Step 2: ARP, finding the MAC address

Ethernet delivers frames by MAC address, but applications use IP addresses. The Address Resolution Protocol (ARP) asks the local network "who has 10.0.1.5?" and the owner replies with its MAC. Results are cached in the ARP table.

```
$ ip neigh show
10.0.1.1 dev eth0 lladdr 02:42:0a:00:01:01 REACHABLE
10.0.1.5 dev eth0 lladdr 02:42:0a:00:01:05 STALE
```

Step 3: VLANs, logical segmentation

A Virtual LAN (VLAN) divides one physical switch into several isolated broadcast domains using a 12-bit tag (IEEE 802.1Q, IDs 1 to 4094). Cloud networks replace VLANs with subnets and security groups, but you meet VLANs in on-premises data centres, Direct Connect and ExpressRoute virtual interfaces.

Step 4: Routing tables

A route table maps a destination prefix to a next hop. The router chooses the **longest prefix match**: the most specific route wins.

```
$ ip route
default via 10.0.1.1 dev eth0
10.0.1.0/24 dev eth0 proto kernel scope link src 10.0.1.25
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
```

Example: a packet to 10.0.1.40 matches 10.0.1.0/24 (24 bits) and default (0 bits). The /24 wins, so the packet is sent directly. A packet to 8.8.8.8 only matches the default route and goes to the gateway.

Step 5: Static and dynamic routing

Type	How routes are learned	Where you meet it
Static	Written by an administrator	VPC route tables, small networks
OSPF	Link-state, within one organisation	On-premises data centre
BGP	Path-vector between autonomous systems	Internet, Direct Connect, ExpressRoute, VPN with dynamic routing, Cilium / Calico BGP mode

BGP essentials for cloud engineers

When you connect on-premises to the cloud with dynamic routing, both sides advertise prefixes with BGP. Know your ASN, the prefixes you advertise, and the AS-path preference. Always filter advertised prefixes so a misconfiguration cannot leak routes.

CHAPTER 5

TCP, UDP and Ports

The transport layer decides how reliably, and how fast, two applications can talk.

Step 1: Ports and sockets

An IP address identifies a host. A **port** (0 to 65535) identifies an application on that host. A connection is uniquely identified by the 5-tuple: *protocol*, *source IP*, *source port*, *destination IP*, *destination port*.

Port	Protocol	Service
22	TCP	SSH
53	UDP / TCP	DNS
80 / 443	TCP	HTTP / HTTPS (443 also carries HTTP/3 over UDP)
123	UDP	NTP time sync
3306	TCP	MySQL
5432	TCP	PostgreSQL
6379	TCP	Redis
6443	TCP	Kubernetes API server
2379-2380	TCP	etcd client and peer
10250	TCP	Kubelet API
30000-32767	TCP / UDP	Kubernetes NodePort range

Step 2: TCP vs UDP

Property	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Delivery	Reliable, ordered, retransmitted	Best effort, no ordering
Overhead	Higher	Lower
Flow / congestion control	Yes	No (application must handle)
Used by	HTTP, SSH, databases, gRPC	DNS, VoIP, video, QUIC / HTTP/3, WireGuard, syslog

Step 3: The TCP three-way handshake

Before any data flows, TCP establishes a connection. A firewall that drops the SYN gives you a **timeout**. A closed port answers with RST and gives you **connection refused**. This difference is the most useful diagnostic clue in networking.

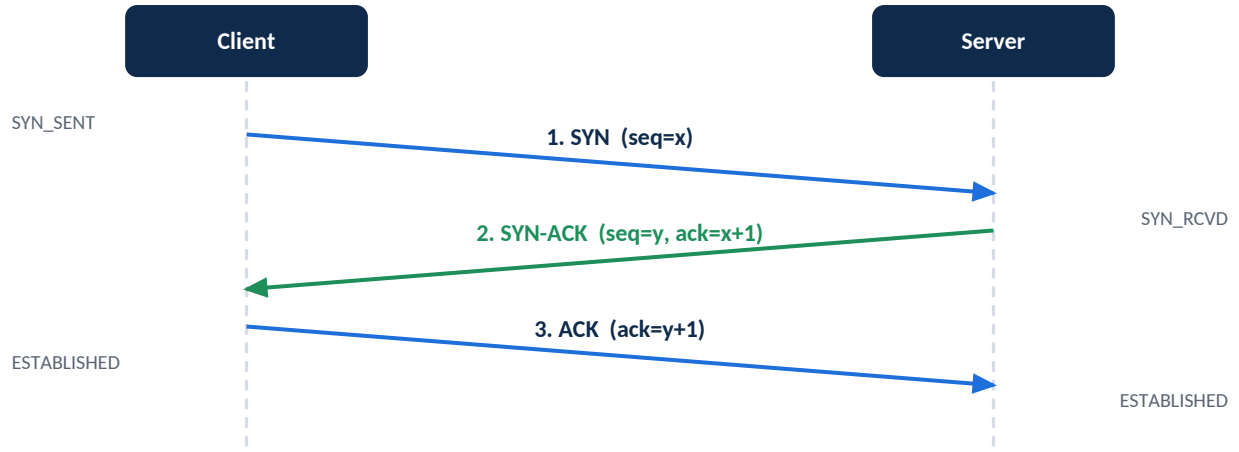


Figure 5.1: TCP three-way handshake.

Step 4: Connection teardown and states

Closing uses FIN packets. The side that closes first enters **TIME_WAIT** for about 60 seconds. High-volume clients that open thousands of short connections can exhaust ephemeral ports. Use connection pooling and keep-alive.

State	Meaning	Typical concern
LISTEN	Server waiting for connections	Missing means the service is not running
ESTABLISHED	Active connection	Count against limits
TIME_WAIT	Waiting after close	Port exhaustion on busy clients
CLOSE_WAIT	Peer closed, app has not	Application is leaking connections
SYN_SENT	Waiting for SYN-ACK	Firewall dropping, wrong route

```
$ ss -tan state established | wc -l           # count established connections
$ ss -tulpn                                  # listening sockets and owning process
$ nc -zv 10.0.2.10 5432                      # test a TCP port
Connection to 10.0.2.10 5432 port [tcp/postgresql] succeeded!
```

CHAPTER 6

DNS, the Name System

DNS is the most common hidden cause of "the network is down". Learn the resolution path and the record types.

Step 1: How a name is resolved

1. The application asks the OS **stub resolver** (reads `/etc/resolv.conf` and the hosts file).
2. The stub resolver asks a **recursive resolver**, in the cloud this is the VPC resolver (AWS: VPC base address + 2).
3. The recursive resolver checks its cache. On a miss it asks a **root server**, then the **TLD server** (.com), then the domain's **authoritative server**.
4. The answer returns with a **TTL**. Every layer caches it until the TTL expires.

Step 2: Record types

Record	Purpose	Example
A	Name to IPv4 address	api.example.com. 300 IN A 203.0.113.10
AAAA	Name to IPv6 address	api.example.com. 300 IN AAAA 2001:db8::10
CNAME	Alias to another name	www IN CNAME lb-123.elb.amazonaws.com.
MX	Mail servers	example.com. IN MX 10 mail.example.com.
TXT	Free text, domain verification, SPF	"v=spf1 include:_spf.example.com ~all"
NS	Delegation to name servers	example.com. IN NS ns1.example.com.
SRV	Service host and port	_sip._tcp IN SRV 10 60 5060 sip.example.com.
PTR	Reverse lookup, IP to name	10.113.0.203.in-addr.arpa. PTR api.example.com.

CNAME restriction

A CNAME cannot exist at the zone apex (example.com itself). Cloud DNS services offer an "alias" record type (Route 53 Alias, Azure alias records) to point the apex at a load balancer.

Step 3: Cloud DNS services

Capability	AWS	Azure	Google Cloud
Public DNS	Route 53 public hosted zone	Azure DNS zone	Cloud DNS public zone
Private DNS	Route 53 private hosted zone	Azure Private DNS zone	Cloud DNS private zone
Hybrid resolution	Route 53 Resolver endpoints	DNS Private Resolver	DNS forwarding / inbound policy
Traffic policies	Weighted, latency, failover, geo	Traffic Manager	Routing policies

Step 4: Debugging DNS

```
$ dig api.example.com +short                # quick answer
$ dig api.example.com A +noall +answer      # answer section with TTL
$ dig @8.8.8.8 api.example.com              # ask a specific resolver
$ dig +trace api.example.com                # follow root -> TLD -> authoritative
$ dig -x 203.0.113.10                       # reverse lookup
$ resolvectl status                         # which resolver does this host use
```

Kubernetes DNS. CoreDNS answers inside the cluster. A Service is reachable as `service.namespace.svc.cluster.local`. The pod's `ndots:5` setting makes short names try several search domains first, which can multiply DNS queries. For external names use a trailing dot or lower ndots when latency matters.

CHAPTER 7

HTTP, TLS and Load Balancing

Most cloud traffic is HTTP over TLS behind a load balancer. This chapter covers each of those pieces.

Step 1: HTTP essentials

Version	Transport	Key feature
HTTP/1.1	TCP	Keep-alive, one request at a time per connection
HTTP/2	TCP + TLS	Multiplexed streams, header compression, used by gRPC
HTTP/3	QUIC over UDP	No head-of-line blocking at transport level, faster connection setup

Status class	Meaning	Operational hint
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirect	301 permanent, 302 temporary, 304 not modified
4xx	Client error	400 bad request, 401 unauthenticated, 403 forbidden, 404 not found, 429 rate limited
5xx	Server error	502 bad gateway (bad upstream response), 503 unavailable, 504 gateway timeout

Step 2: TLS, how encryption is established

Transport Layer Security protects data in transit and proves the server's identity with a certificate. TLS 1.3 completes the handshake in one round trip.

1. **ClientHello:** client sends supported versions, cipher suites and a key share.
2. **ServerHello:** server picks parameters, sends its key share, certificate and proof of key ownership.
3. The client **verifies the certificate chain**, hostname and expiry against trusted certificate authorities.
4. Both sides derive session keys. Application data is now encrypted.

```
$ openssl s_client -connect example.com:443 -servername example.com </dev/null 2>/dev/null \
    | openssl x509 -noout -subject -issuer -dates
$ curl -vI https://example.com               # headers, TLS version and certificate info
```

Certificate hygiene

Expired certificates are among the most common causes of production outages. Automate issuance and renewal (ACM, cert-manager with Let's Encrypt) and alert at least 21 days before expiry. Use TLS 1.2 as the minimum and prefer TLS 1.3.

Step 3: Layer 4 vs Layer 7 load balancers

	Layer 4 (transport)	Layer 7 (application)
Sees	IP, port, protocol	HTTP host, path, headers, cookies
Speed	Very high throughput, low latency	Slightly higher latency, richer features
Features	Static IP, TCP / UDP / TLS passthrough	Path routing, redirects, WAF, auth, gRPC
AWS	Network Load Balancer	Application Load Balancer
Azure	Azure Load Balancer	Application Gateway, Front Door
Google Cloud	Network Load Balancer	Application Load Balancer
Pick when	Non-HTTP protocols, extreme performance, fixed IPs	Microservices routing, web apps, APIs

Step 4: Health checks and algorithms

A load balancer only sends traffic to healthy targets. Configure the health check path to verify real readiness (for example database connectivity) but keep it cheap. Common algorithms:

- **Round robin:** rotate through targets, simple and fair for uniform requests.
- **Least connections / least outstanding requests:** favour the least busy target, best for uneven request cost.
- **IP hash / consistent hash:** same client reaches the same target, useful for caches.
- **Weighted:** shift a percentage of traffic, used for canary and blue-green releases.

Connection draining (deregistration delay) lets in-flight requests finish before a target is removed. Set it to at least your longest normal request, otherwise every deployment drops requests.

CHAPTER 8

NAT, Firewalls and Access Control

Decide who may talk to whom, and how private hosts reach the Internet without being reachable from it.

Step 1: Network Address Translation

Type	What it does	Cloud example
Source NAT (SNAT)	Rewrites the source IP so private hosts share a public IP for outbound traffic	AWS NAT Gateway, Azure NAT Gateway, Cloud NAT
Destination NAT (DNAT)	Rewrites the destination so inbound traffic reaches a private host	Port forwarding, load balancers, Kubernetes NodePort
Static 1:1 NAT	Maps one public IP to one private IP	AWS Elastic IP on an instance

NAT gateway cost and limits

NAT gateways charge per hour and per GB processed. Heavy traffic to S3, container registries or other cloud services should use gateway or interface endpoints to avoid both cost and the bottleneck. Place one NAT gateway per Availability Zone to survive an AZ failure.

Step 2: Stateful vs stateless filtering

A **stateful** firewall remembers connections. If you allow outbound traffic, the reply is automatically allowed. A **stateless** filter evaluates every packet independently, so you must allow both directions explicitly, including ephemeral return ports (1024-65535).

	Security Group (AWS)	Network ACL (AWS)	Azure NSG	GCP Firewall rule
Attached to	Network interface	Subnet	Subnet or NIC	VPC network, targets by tag / service account
State	Stateful	Stateless	Stateful	Stateful
Rule types	Allow only	Allow and deny	Allow and deny	Allow and deny
Evaluation	All rules together	In number order	By priority	By priority
Reference by	Other security groups	CIDR only	Service tags, ASGs	Tags, service accounts

Step 3: Writing least-privilege rules

1. Start from **deny all**. Add only the flows the architecture requires.
2. Reference **security groups instead of IP ranges**: allow the app tier's group to reach the database port, not "10.0.0.0/8".
3. Never expose management ports (22, 3389) to 0.0.0.0/0. Use a bastion with identity-aware access, SSM Session Manager, or Azure Bastion.
4. Keep rules **documented and tagged**, and review for unused rules every quarter.
5. Enable **flow logs** so you can see accepted and rejected traffic.

CHAPTER 9

Cloud Networking

The same ideas apply on every cloud, only the names change. Learn the pattern, then map it to your provider.

Step 1: Building blocks and vocabulary

Concept	AWS	Azure	Google Cloud
Private network	VPC	Virtual Network (VNet)	VPC network (global)
Subnet scope	One Availability Zone	Regional, spans zones	Regional, spans zones
Internet access in	Internet Gateway + public IP	Public IP	External IP
Private outbound	NAT Gateway	NAT Gateway	Cloud NAT
Private access to services	VPC endpoints, PrivateLink	Private Endpoint, Service Endpoint	Private Service Connect, Private Google Access
VPC to VPC	VPC Peering, Transit Gateway	VNet Peering, Virtual WAN	VPC Peering, Network Connectivity Center
Dedicated private link	Direct Connect	ExpressRoute	Cloud Interconnect
Encrypted VPN	Site-to-Site VPN	VPN Gateway	Cloud VPN

Step 2: Reference architecture, a multi-AZ VPC

Production workloads are normally split into **public subnets** (load balancers, NAT gateways) and **private subnets** (application servers, databases), repeated in at least two Availability Zones for resilience.

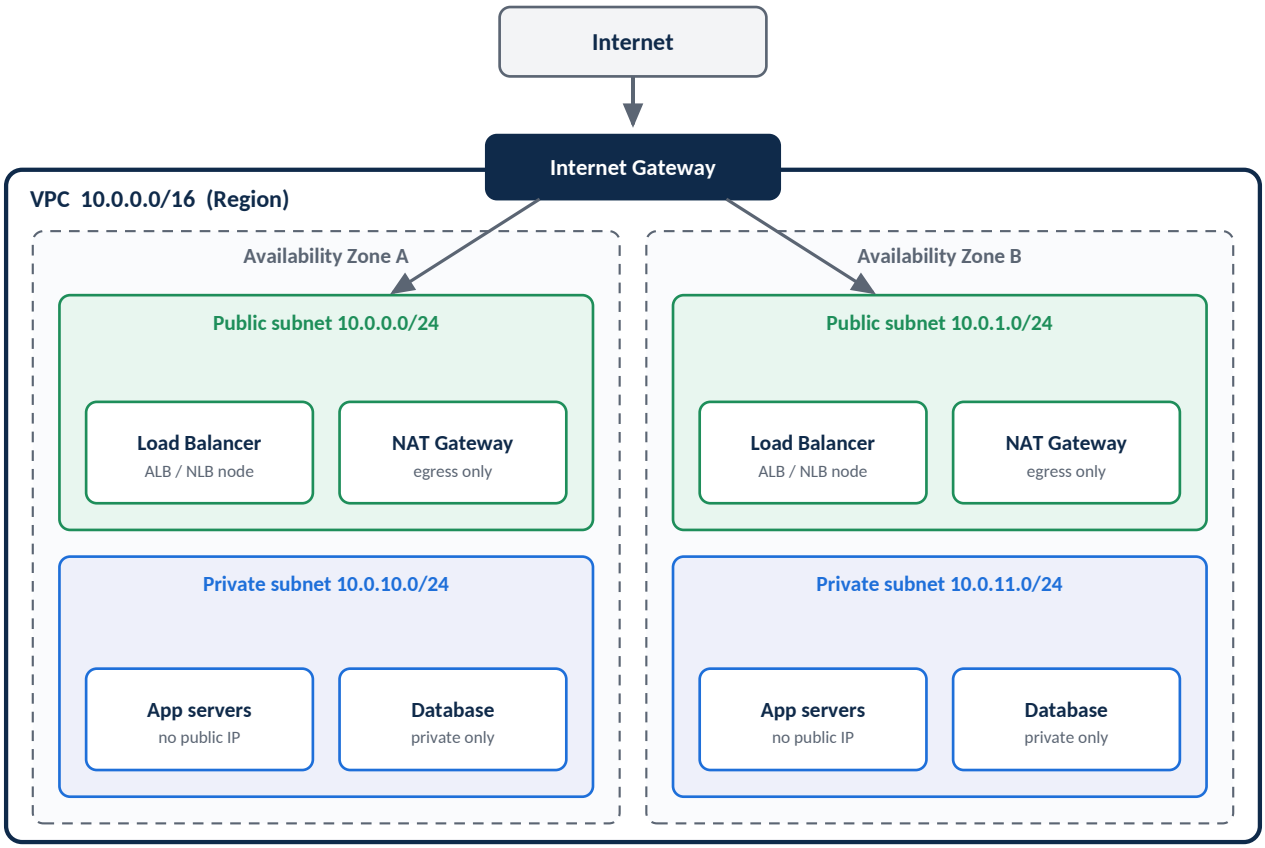


Figure 9.1: Two-AZ VPC with public and private subnets.

Step 3: Build it, in order

1. **Choose the VPC CIDR** from the organisation plan (Chapter 3), for example 10.10.0.0/16.
2. **Create subnets** per tier and per AZ. Leave unallocated space for future tiers.
3. Attach an **Internet Gateway** and create a public route table with `0.0.0.0/0 to igw`.
4. Create a **NAT Gateway in each public subnet**. Create one private route table per AZ with `0.0.0.0/0 to nat` of the same AZ.
5. Add **VPC endpoints** for S3 and DynamoDB (gateway type, free) and for services such as ECR, SSM and Secrets Manager (interface type).
6. Define **security groups** per tier and reference them rather than CIDRs.
7. Turn on **VPC Flow Logs** and DNS query logging.
8. Deploy the **load balancer** in public subnets and targets in private subnets.

Step 4: Connecting networks

Option	Best for	Transitive?	Watch out for
VPC Peering	A few one-to-one connections	No	CIDRs must not overlap; mesh grows as n(n-1)/2
Transit Gateway / Virtual WAN	Hub-and-spoke for many VPCs and on-premises	Yes	Per-attachment and per-GB cost; route table design
PrivateLink / Private Endpoint	Expose one service privately, overlap-safe	n/a (service-level)	One-way: consumer to provider only
Site-to-Site VPN	Quick encrypted link over the Internet	Depends on gateway	Throughput limits per tunnel, use two tunnels
Direct Connect / ExpressRoute / Interconnect	Stable, high-bandwidth private link	Via gateway	Provisioning takes weeks; add VPN as backup

Design rules that prevent pain later

1) Never overlap CIDRs across any network you might connect. 2) Size subnets generously, especially for Kubernetes. 3) Keep databases and internal services in private subnets with no public IP. 4) Prefer endpoints over NAT for cloud service traffic. 5) Build everything in at least two AZs.

Step 5: A minimal Terraform VPC

```
resource "aws_vpc" "main" {
  cidr_block           = "10.10.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = "prod-vpc" }
}

resource "aws_subnet" "public_a" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.10.0.0/24"
  availability_zone       = "ap-south-1a"
  map_public_ip_on_launch = true
  tags = { Name = "public-a" }
}

resource "aws_subnet" "private_a" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.10.10.0/24"
  availability_zone = "ap-south-1a"
  tags = { Name = "private-a" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}

resource "aws_route_table_association" "public_a" {
  subnet_id      = aws_subnet.public_a.id
  route_table_id = aws_route_table.public.id
}
```

Least-privilege rule: only the app tier's security group may reach the database port (referenced by group, not by CIDR).

```
# Example: allow only the app tier to reach PostgreSQL (Terraform)
resource "aws_security_group_rule" "db_from_app" {
  type                     = "ingress"
  from_port                = 5432
  to_port                  = 5432
  protocol                 = "tcp"
  security_group_id        = aws_security_group.db.id
  source_security_group_id = aws_security_group.app.id
}
```

CHAPTER 10

Kubernetes Networking

Kubernetes adds a software-defined network on top of the cloud network. Four rules describe all of it.

Step 1: The Kubernetes network model

Kubernetes requires a flat network where:

- Every **Pod gets its own IP address**.
- Every Pod can reach every other Pod **without NAT**, across nodes.
- Agents on a node (kubelet) can reach all Pods on that node.
- Services give a stable virtual IP and DNS name to a changing set of Pods.

Step 2: CNI plugins

The Container Network Interface (CNI) plugin implements the Pod network. Your choice affects performance, IP consumption and available security features.

CNI	Model	Strengths	Consider
AWS VPC CNI	Pods get real VPC IPs	Native VPC routing, security groups for pods	Consumes many subnet IPs; plan large subnets
Azure CNI / Overlay	VNet IPs or overlay	Native Azure integration	Overlay mode saves IPs
Calico	BGP routing or VXLAN / IPIP overlay	Mature NetworkPolicy, scalable	Overlay adds header overhead
Cilium	eBPF dataplane	High performance, L7 policy, observability (Hubble)	Needs modern kernel
Flannel	Simple VXLAN overlay	Easy to run	No NetworkPolicy by itself

Step 3: Services, stable access to Pods

Type	Reachable from	How it works
ClusterIP	Inside the cluster	Virtual IP load-balanced to ready Pods by kube-proxy or eBPF
NodePort	Node IP + port 30000-32767	Opens a port on every node and forwards to the Service
LoadBalancer	Internet or VPC	Provisions a cloud load balancer pointing at the Service
ExternalName	Inside the cluster	DNS CNAME to an external hostname
Headless (clusterIP: None)	Inside the cluster	DNS returns Pod IPs directly, used by StatefulSets

Step 4: Following a request end to end

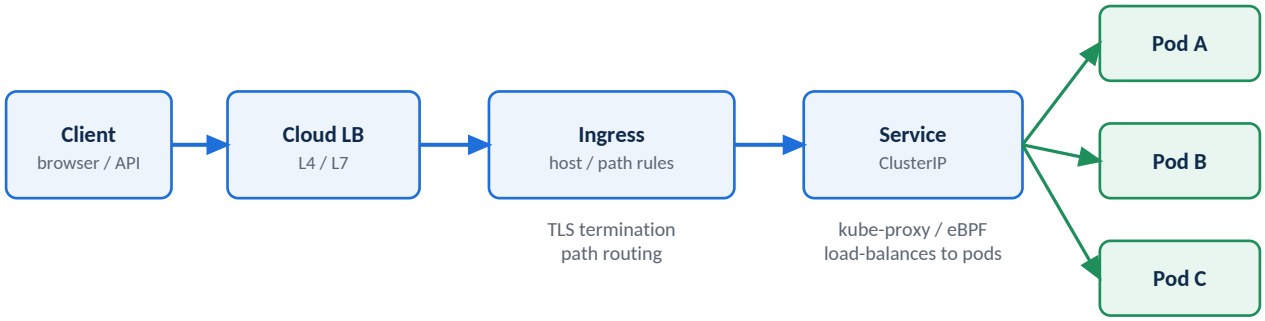


Figure 10.1: Path of an external request to a Pod.

1. The client resolves the hostname via DNS to the cloud load balancer.
2. The cloud load balancer forwards to the Ingress controller (or Gateway) running as Pods or nodes.
3. The Ingress controller terminates TLS and matches host and path rules.
4. It forwards to a Service. kube-proxy iptables / IPVS rules or Cilium eBPF pick one ready Pod.
5. The packet reaches the Pod over the CNI network and the app responds along the same path.

Step 5: Ingress and Gateway API

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: nginx
  tls:
  - hosts: ["app.example.com"]
    secretName: app-tls
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service: { name: api, port: { number: 80 } }
      - path: /
        pathType: Prefix
        backend:
          service: { name: web, port: { number: 80 } }
```

The newer **Gateway API** (Gateway, HTTPRoute) separates infrastructure ownership from application routing and is the recommended direction for new clusters.

Step 6: NetworkPolicy, default deny

By default every Pod can talk to every other Pod. Apply a default-deny policy per namespace, then allow specific flows. NetworkPolicy only works if your CNI enforces it (Calico, Cilium, Azure Network Policy).

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny, namespace: prod }
spec:
  podSelector: {}
  policyTypes: ["Ingress", "Egress"]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-web-to-api, namespace: prod }
spec:
  podSelector: { matchLabels: { app: api } }
  policyTypes: ["Ingress"]
  ingress:
  - from:
    - podSelector: { matchLabels: { app: web } }
    ports:
    - { protocol: TCP, port: 8080 }
```

Remember DNS in default deny

After denying egress, Pods can no longer resolve names. Add an egress rule allowing UDP and TCP port 53 to the kube-dns / CoreDNS Pods.

Avoid CIDR overlap in clusters

The node network, the Pod CIDR and the Service CIDR must all be distinct, and distinct from any network you will peer with or reach over VPN. Changing these ranges later usually means rebuilding the cluster.

CHAPTER 11

Network Security and Zero Trust

Perimeter-only security fails the moment one workload is compromised. Layer your defences.

Step 1: Defence in depth

Layer	Control	Tools
Edge	DDoS protection, WAF, rate limiting	AWS Shield / WAF, Azure Front Door WAF, Cloud Armor, Cloudflare
Network	Segmentation, private subnets, security groups, NACLs	VPC design, NSGs, firewall rules
Workload	Pod and host policies, runtime detection	NetworkPolicy, Cilium, Falco
Identity	Strong authentication between services	IAM roles, mTLS, SPIFFE / SPIRE, service mesh
Data	Encryption in transit and at rest	TLS 1.2+, KMS, Key Vault

Step 2: Zero trust principles

- **Never trust the network location.** Being inside the VPC grants nothing by itself.
- **Authenticate and authorise every request**, user and service, with short-lived credentials.
- **Least privilege** for both IAM and network paths.
- **Encrypt everything in transit**, including internal east-west traffic (mTLS).
- **Log and monitor continuously** and assume breach.

Step 3: Secure remote access

- Replace public SSH with **SSM Session Manager**, **Azure Bastion** or **IAP TCP forwarding**.
- Use a **VPN or identity-aware proxy** (WireGuard, Tailscale, Zero Trust access products) for admin tools.
- Place databases and Kubernetes API servers behind private endpoints or restricted CIDRs.
- Rotate credentials and prefer certificate or IAM-based login over passwords.

Step 4: Service mesh and mTLS

A service mesh (Istio, Linkerd, Cilium service mesh) adds mutual TLS, retries, timeouts, traffic splitting and telemetry between services without changing application code. It is powerful but adds operational weight. Adopt it when you need uniform mTLS and traffic control across many services, not by default.

Step 5: Visibility

- **Flow logs** (VPC Flow Logs, NSG flow logs, VPC flow logs in GCP) for who talked to whom.
- **DNS query logs** to detect command-and-control and data exfiltration.
- **Load balancer access logs** for request-level analysis.
- **Hubble, eBPF or packet mirroring** for deep inspection when needed.

CHAPTER 12

Troubleshooting Method and Toolkit

Good engineers do not guess. They follow the path of the packet and test one layer at a time.

Step 1: A repeatable method

1. **Define the symptom precisely.** From where to where? Which protocol and port? Always, or intermittent? Since when?
2. **Check what changed.** Deployments, security group edits, route changes, certificate renewals, DNS updates.
3. **Work bottom-up or follow the path.** Name resolution, then routing, then firewall, then the listening service, then the application.
4. **Test from both ends.** A client-side test proves the path; a server-side capture proves the packet arrived.
5. **Change one thing at a time** and record the result.
6. **Document the root cause** and add monitoring so it is caught earlier next time.

Step 2: The command toolkit

Question	Command	What to look for
What is my IP and interface state?	`ip addr`	Interface UP, expected address and MTU
What is my route to the target?	`ip route get 10.0.2.10`	Correct next hop and interface
Is the host alive?	`ping -c 4 10.0.2.10`	Replies and latency. ICMP may be blocked, so absence is not proof
Where does the path break?	`mtr -rw 10.0.2.10`	Loss or latency jump starting at one hop
Does the name resolve?	`dig app.example.com`	Correct answer, TTL, resolver used
Is the port reachable?	`nc -zv host 443`	succeeded / refused / timed out
What does the HTTP exchange look like?	`curl -v https://host/path`	Status, headers, TLS errors, redirect loops
Is my service listening?	`ss -tulpn`	Bound to 0.0.0.0 or the right IP, not only 127.0.0.1
What is on the wire?	`tcpdump -ni eth0 host 10.0.2.10 and port 443`	SYN without SYN-ACK, RST, retransmissions
Is the certificate valid?	`openssl s_client -connect host:443`	Chain, expiry, hostname match

Step 3: Reading the symptom

Symptom	Most likely cause	First check
Connection timed out	Firewall / security group dropping, no route, wrong IP	Security groups, NACLs, route table, flow logs
Connection refused	Packet arrived but nothing listens on the port	`ss -tulpn` on the server, service status, bind address
Name not resolved	DNS record missing, wrong resolver, blocked UDP/TCP 53	`dig` against the VPC resolver and a public one
Works by IP, fails by name	DNS problem, split-horizon mismatch, SNI / Host header	Resolver in use, private hosted zone association
Small requests work, large hang	MTU / fragmentation, blocked ICMP "fragmentation needed"	Test with `ping -M do -s 1400`, lower MTU, allow ICMP type 3
Intermittent timeouts	Unhealthy target, conntrack table full, NAT port exhaustion, one bad AZ	LB target health, NAT gateway metrics, node conntrack stats
502 Bad Gateway	Upstream closed connection or crashed	Application logs, keep-alive timeout mismatch
504 Gateway Timeout	Upstream too slow or unreachable from LB	LB idle timeout vs app timeout, health, security group from LB to target
TLS handshake failure	Expired / wrong certificate, protocol or cipher mismatch, wrong SNI	`openssl s_client` with `-servername`
Pod cannot reach external site	NetworkPolicy egress, missing NAT route, DNS ndots delay	Test from the node, check policy and route of the node subnet

> **Timeout vs refused**
>
> This one distinction narrows the search enormously. **Refused** means the network path works and the host answered with a reset: fix the service or its port. **Timeout** means packets vanished on the way: fix routing or firewalls.

Step 4: Walkthrough, "the app cannot reach the database"

1. From the app host: `dig db.internal.example.com`. It should return a private IP. If not, fix DNS or the private hosted zone association.
2. Run `ip route get <db-ip>`. The route should be local to the VPC. If it points to a NAT or nowhere, correct the route table.
3. Run `nc -zv <db-ip> 5432`. **Timed out**: inspect the database security group inbound rule (source must be the app security group) and any NACL.
4. **Refused**: on the database host run `ss -tulpn | grep 5432`. Check it listens on the private IP, not 127.0.0.1.
5. If the port opens but login fails, the network is fine. Check credentials, TLS requirements and `pg_hba.conf`.

Step 5: Walkthrough, "the Kubernetes Service has no response"

```
$ kubectl get svc,endpoints api -n prod          # endpoints must not be empty
$ kubectl get pods -n prod -l app=api -o wide    # pods Ready? which node and IP?
$ kubectl describe svc api -n prod               # selector matches pod labels?
$ kubectl run -it --rm dbg --image=nicolaka/netshoot -- bash
  # inside the debug pod:
  dig api.prod.svc.cluster.local
  curl -v http://api.prod.svc.cluster.local:80/health
$ kubectl get networkpolicy -n prod              # is something denying the flow?
$ kubectl logs -n ingress-nginx deploy/ingress-nginx-controller | tail
```

Empty endpoints almost always mean a selector that does not match the Pod labels, or Pods failing their readiness probe.

Step 6: Metrics worth alerting on

- Load balancer 5xx rate, target response time and unhealthy host count.
- NAT gateway `ErrorPortAllocation` and packet drop counters.
- DNS resolution failures and latency, CoreDNS error rate.
- TCP retransmits and connection resets per node.
- Certificate days-to-expiry and VPN / Direct Connect tunnel state.
- Conntrack table usage on Kubernetes nodes.

CHAPTER 13

Practice Labs and Review Checklists

Reading builds recognition. Building builds skill. Use these labs and checklists to turn this guide into habit.

Step 1: Hands-on labs

Lab	Goal	Proves you understand
1. Subnet it	Split 10.20.0.0/16 into 8 equal subnets by hand, then verify with a calculator	CIDR and subnet math
2. Trace a packet	Run `tcpdump` while you `curl` a site and identify DNS, handshake, TLS and HTTP	Layers and encapsulation
3. Break DNS	Point a record to a wrong IP, observe caching with TTL, then fix it	Resolution and TTL
4. Private VPC	Build a two-AZ VPC with Terraform: public ALB, private app, private DB	Cloud design pattern
5. Lock it down	Replace CIDR rules with security group references, close SSH, add SSM access	Least privilege
6. Cluster policy	Apply default-deny NetworkPolicy and allow only web to api	Kubernetes segmentation
7. Chaos test	Block one AZ's NAT route and watch traffic, alarms and recovery	Resilience

Step 2: Network design review checklist

Area	Question	Done
Addressing	CIDRs are unique across all clouds, regions and on-premises?	[ ]
Addressing	Subnets sized for growth, including Kubernetes pod IP demand?	[ ]
Resilience	Every tier spans at least two Availability Zones?	[ ]
Resilience	One NAT gateway per AZ, and a backup path for VPN / dedicated links?	[ ]
Security	Databases and internal services have no public IP?	[ ]
Security	No management port open to the whole Internet?	[ ]
Security	Rules reference security groups, not broad CIDRs?	[ ]
Security	TLS enforced everywhere, certificates auto-renewed?	[ ]
Cost	Cloud-service traffic uses endpoints instead of NAT?	[ ]
Cost	Cross-AZ and cross-region data transfer understood and minimised?	[ ]
Operations	Flow logs, DNS logs and load balancer logs enabled?	[ ]
Operations	Everything defined as code and peer reviewed?	[ ]
Operations	Runbooks and alerts exist for DNS, LB health, NAT and certificates?	[ ]

Step 3: Common mistakes to avoid

- Choosing a small or default CIDR (for example 172.31.0.0/16) and discovering the overlap when you need to peer.
- Running a single NAT gateway or a single AZ and calling it production.
- Opening 0.0.0.0/0 "temporarily" and forgetting it.
- Setting load balancer idle timeout higher than the application keep-alive, causing random 502 errors.
- Ignoring DNS TTL when planning a cutover or failover.
- Assuming ping failing means the host is down. ICMP is often blocked on purpose.
- Skipping flow logs, then having nothing to investigate with during an incident.

Step 4: Where to go next

- Capture traffic with Wireshark and read real handshakes, DNS replies and TLS records.
- Study your cloud provider's networking certification material (AWS Advanced Networking Specialty, Azure Network Engineer, Google Professional Cloud Network Engineer).
- Read the Kubernetes networking documentation and the Cilium or Calico guides for your CNI.
- Learn an observability stack for network telemetry: Prometheus, Grafana, OpenTelemetry, Hubble.

CHAPTER 14

Glossary

Quick definitions for the terms used in this guide.

Term	Definition
ACL	Access Control List, a list of allow or deny rules.
ALB / NLB	Application / Network Load Balancer (AWS), layer 7 and layer 4 respectively.
ARP	Address Resolution Protocol, maps an IP address to a MAC address on a local network.
AS / ASN	Autonomous System and its number, a network under one routing policy used by BGP.
AZ	Availability Zone, an isolated data-centre location within a region.
BGP	Border Gateway Protocol, the routing protocol of the Internet and of hybrid cloud links.
CIDR	Classless Inter-Domain Routing, prefix notation such as 10.0.0.0/16.
CNI	Container Network Interface, the plugin standard for Kubernetes pod networking.
CoreDNS	The DNS server that runs inside Kubernetes clusters.
DNAT / SNAT	Destination / Source Network Address Translation.
DNS	Domain Name System, translates names to IP addresses.
eBPF	Kernel technology that runs sandboxed programs for fast networking, security and observability.
Egress / Ingress	Outbound / inbound traffic relative to a boundary. In Kubernetes, Ingress is also a routing resource.
Gateway	A device or service that connects networks, such as an Internet Gateway or default gateway.
ICMP	Internet Control Message Protocol, used by ping and error signalling.
MAC address	Hardware address of a network interface used at layer 2.
mTLS	Mutual TLS, both client and server authenticate with certificates.
MTU	Maximum Transmission Unit, the largest packet size a link carries without fragmenting.
NAT	Network Address Translation, rewriting addresses as packets cross a boundary.
NSG	Network Security Group (Azure).
Overlay network	A virtual network tunnelled over another network, for example VXLAN.
Peering	A direct private connection between two virtual networks.
QUIC	UDP-based transport that underlies HTTP/3.
SNI	Server Name Indication, the hostname a client announces during the TLS handshake.
Subnet	A smaller network carved out of a larger address range.
TTL (DNS)	Time to live, how long an answer may be cached. (IP TTL is a hop limit.)
VLAN	Virtual LAN, logical layer 2 segmentation using 802.1Q tags.
VPC / VNet	Virtual Private Cloud / Virtual Network, your isolated network in the cloud.
VPN	Virtual Private Network, an encrypted tunnel, often IPsec or WireGuard.
VXLAN	Tunnelling protocol that carries layer 2 frames over UDP.
WAF	Web Application Firewall, filters HTTP attacks such as SQL injection.

CHAPTER 15

Interview Questions and Answers

Common questions for cloud, DevOps and SRE roles, with concise model answers based on this guide. Add a real example from your own work to each answer.

Networking fundamentals

Q1. What happens when you type a URL in the browser and press Enter?

The browser resolves the name through DNS (cache, then recursive resolver). It opens a TCP connection with the three-way handshake, then a TLS handshake to verify the certificate and agree on keys. It sends the HTTP request, often through a CDN or load balancer to a backend, receives the response and renders the page.

Q2. Explain the OSI model and why it is useful.

It splits communication into seven layers: physical, data link, network, transport, session, presentation, application. It gives a shared vocabulary ("layer 4 load balancer") and lets you isolate a fault: DNS and HTTP are layer 7, ports and TCP are layer 4, IP and routing are layer 3.

Q3. What is the difference between TCP and UDP? When would you choose UDP?

TCP is connection-oriented and reliable: handshake, ordering, retransmission, flow and congestion control. UDP is connectionless with no delivery guarantee but low overhead. Choose UDP for DNS, real-time voice and video, and QUIC / HTTP/3, where the application handles loss.

Q4. What is the difference between "connection refused" and "connection timed out"?

Refused means the packet reached the host and was answered with a reset: the path works but nothing listens on that port. Timed out means packets were dropped on the way, usually by a firewall, security group or missing route. This single clue decides where to look first.

Q5. How many usable hosts are in a /26, and how do you calculate it?

A /26 leaves 6 host bits: 2^6 = 64 addresses, so 62 usable in a classic network (minus network and broadcast). In AWS, 5 addresses are reserved per subnet, so 59 are usable. Always check the provider's reserved count.

Q6. What is NAT, and why do private subnets need a NAT gateway?

NAT rewrites addresses as packets cross a boundary. Source NAT lets private instances share one public IP for outbound traffic. Private instances have no public IP, so a NAT gateway in a public subnet lets them reach the Internet for updates without being reachable from it.

Q7. Explain longest prefix match.

When several routes match a destination, the router uses the most specific one, the longest prefix. A packet to 10.0.1.40 matches 10.0.1.0/24 and 0.0.0.0/0; the /24 wins. This is how cloud route tables and routers decide the next hop.

Q8. What are common DNS record types, and what does TTL do?

A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), TXT (verification, SPF), NS (delegation), PTR (reverse), SRV (service and port). TTL is how long resolvers may cache an answer, so lower it before a planned cutover. A CNAME cannot sit at the zone apex; use an alias record.

Q9. What is the difference between a Layer 4 and a Layer 7 load balancer?

Layer 4 (NLB, Azure Load Balancer) forwards TCP/UDP by IP and port with very high throughput and static IPs. Layer 7 (ALB, Application Gateway) understands HTTP and routes by host, path and headers, terminates TLS and integrates with a WAF. Use L7 for web apps and microservices, L4 for other protocols.

Q10. Stateful vs stateless firewalls: what is the difference between a security group and a network ACL?

A stateful firewall tracks connections, so return traffic is allowed automatically (security groups, Azure NSGs). A stateless one evaluates each packet alone, so both directions and ephemeral ports must be allowed (AWS NACLs). Security groups attach to NICs and allow only; NACLs attach to subnets and support deny.

Cloud networking

Q11. How do you design a secure, highly available three-tier VPC?

Pick a non-overlapping CIDR, then create public subnets (load balancer, NAT) and private subnets (app, database) in at least two AZs. Use an Internet Gateway for public routes, one NAT gateway per AZ for private egress, security groups that reference each other, VPC endpoints for cloud services, flow logs, and define it all as code.

Q12. What makes a subnet "public"?

Its route table has a default route (0.0.0.0/0) to an Internet Gateway, and its instances have public IPs. A private subnet has no such route; at most it routes outbound through a NAT gateway.

Q13. VPC peering or Transit Gateway?

Peering is simple, cheap and one-to-one but not transitive, and a full mesh grows as n(n-1)/2. Transit Gateway (or Azure Virtual WAN) is a hub for many VPCs and on-premises links with centralised routing, at extra per-attachment and per-GB cost. Use peering for a few links, a hub for many.

Q14. Site-to-Site VPN or Direct Connect / ExpressRoute?

A VPN is quick to set up, encrypted and runs over the Internet, with variable latency and per-tunnel limits. Direct Connect / ExpressRoute is a dedicated private link with consistent latency and high bandwidth but takes weeks to provision. Many teams use the dedicated link as primary with VPN as backup.

Q15. How can private instances reach S3 without going through the Internet or NAT?

Use a VPC gateway endpoint for S3 (and DynamoDB): it adds a route to the service inside the AWS network, is free and avoids NAT processing charges. Interface endpoints (PrivateLink) do the same for other services such as ECR, SSM and Secrets Manager.

Q16. What is PrivateLink, and why does it help with overlapping CIDRs?

It exposes a service from a provider VPC to consumer VPCs through a private endpoint (an ENI in the consumer subnet). Traffic is one-way, consumer to provider, and no routes between the networks are needed, so overlapping address ranges do not matter.

Q17. How do you prevent IP address conflicts across many accounts and clouds?

Keep a central IP address plan (IPAM) assigning each environment and region a unique block from a large range such as 10.0.0.0/8, reserve space for growth and on-premises, and enforce it through code review and automation. Fixing overlaps later is painful.

Kubernetes networking

Q18. What are the rules of the Kubernetes network model?

Every pod gets its own IP; every pod can reach every other pod without NAT, across nodes; node agents can reach all pods on the node. Services give a stable virtual IP and DNS name to a changing set of pods. A CNI plugin implements the model.

Q19. Explain the Service types.

ClusterIP is internal only. NodePort opens a port (30000-32767) on every node. LoadBalancer provisions a cloud load balancer. ExternalName returns a DNS CNAME. A headless Service (clusterIP: None) returns pod IPs directly, used by StatefulSets.

Q20. Trace a request from the Internet to a pod.

DNS resolves to the cloud load balancer, which forwards to the Ingress controller. It terminates TLS and matches host and path rules, then sends the request to a Service. kube-proxy (iptables / IPVS) or eBPF picks a ready pod and the CNI network delivers the packet.

Q21. Overlay networking versus native VPC networking for pods?

Overlays (VXLAN, IPIP) tunnel pod traffic over the node network: easy and IP-efficient, but with header overhead and extra debugging. Native modes (AWS VPC CNI, Calico BGP) give pods routable IPs with better performance and visibility but consume many subnet addresses, so plan large subnets.

Q22. What is a default-deny NetworkPolicy, and what is the common gotcha?

It selects all pods in a namespace and allows no traffic; you then allow specific flows. The gotcha is egress: after denying it, pods cannot resolve DNS, so allow port 53 to CoreDNS. NetworkPolicy also works only if the CNI enforces it (Calico, Cilium).

Q23. Ingress, LoadBalancer Service and Gateway API: how do they differ?

A LoadBalancer Service creates one cloud load balancer per Service at layer 4. Ingress shares one entry point and routes by host and path at layer 7. Gateway API is the newer, more expressive model that separates infrastructure (Gateway) from routing (HTTPRoute) and is the direction for new clusters.

Troubleshooting scenarios

Q24. An application cannot connect to its database. How do you debug it?

Work along the path: (1) `dig` the hostname for a private IP; (2) `ip route get` to confirm the route; (3) `nc -zv host 5432`. A timeout points to the database security group, NACL or routing; a refusal means check `ss -tulpn` and the bind address on the server; if the port opens, check credentials and TLS.

Q25. Users see random 502 errors right after deployments. What do you check?

Targets removed without connection draining, pods receiving traffic before they are ready, or the backend keep-alive timeout being shorter than the load balancer idle timeout, so it closes connections the balancer still reuses. Fix with readiness probes, a preStop delay, deregistration delay and keep-alive longer than the balancer idle timeout.

Q26. Small requests work but large uploads hang. What is the likely cause?

An MTU or fragmentation problem: tunnels (VPN, VXLAN) shrink the usable size and the ICMP "fragmentation needed" message is blocked. Test with `ping -M do -s 1400`, lower the MTU or MSS, and allow ICMP type 3.

Q27. A Kubernetes Service returns no response. What are your steps?

Run `kubectl get endpoints`: empty endpoints mean the selector does not match pod labels or pods fail readiness. Then test from a debug pod (DNS and curl against the service name), review NetworkPolicies, and check the Ingress controller logs.

Q28. How would you apply zero trust principles to a cloud network?

Do not trust location: authenticate and authorise every request with short-lived identity (IAM roles, mTLS), allow only required flows with least-privilege security groups and NetworkPolicies, remove public SSH in favour of SSM or a bastion, encrypt traffic in transit, and log flows and DNS to detect anomalies.

> **Interview tip**
>
> Structure answers as: concept, then how it works in practice, then a trade-off or a real example. Interviewers value the troubleshooting method (symptom, layer, test, fix) more than memorised commands.